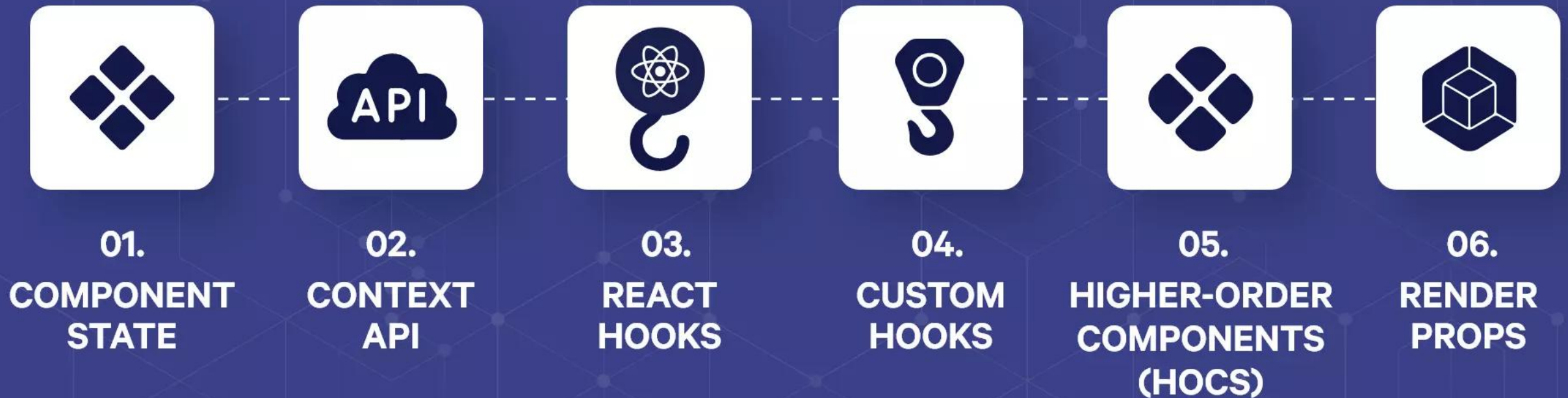


ІНСТРУМЕНТИ REACT ДЛЯ КЕРУВАННЯ СТАНОМ

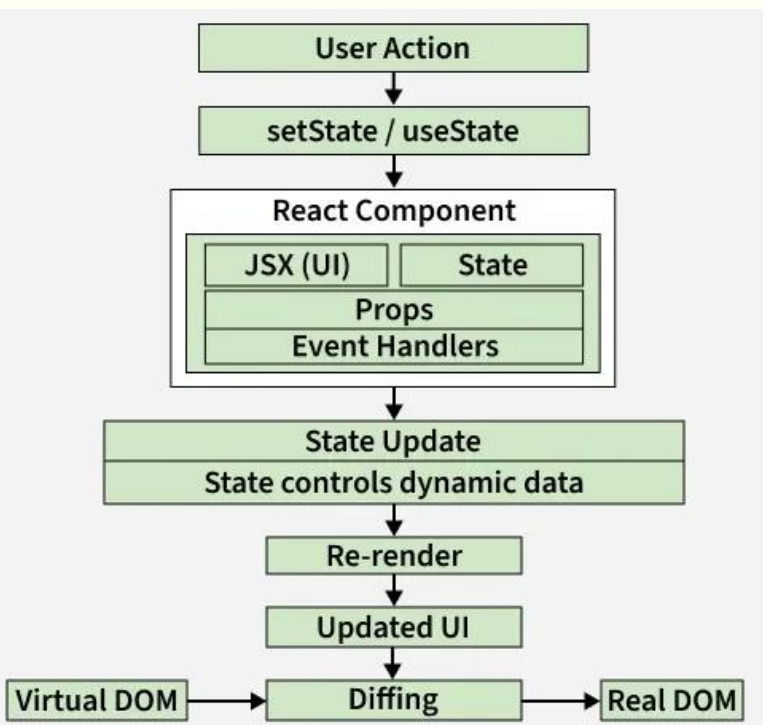
Питання 5.2.

Різні підходи до керування станом у React



- У React стан – це вбудований об'єкт JavaScript, який використовується для зберігання та керування динамічними даними в компоненті.
 - Стан представляє дані, які можуть змінюватися з часом та впливати на поведінку та рендеринг компонента.
 - Оновлюючи стан, компоненти React можуть ініціювати повторний рендеринг.

Керування станом компонента



- Компоненти React – це повторно використовувані одиниці інтерфейсу, які обробляють власну логіку, приймають дані через властивості, керують станом та ефективно оновлюють лише ті частини інтерфейсу, які змінюються.

- Дія користувача запускає подію (кляцання, введення тощо).
- `setState / useState` оновлює стан компонента.
- Компонент React використовує `props`, `state`, `JSX` та обробники подій для визначення логіки інтерфейсу користувача.
- Оновлення стану змінює динамічні дані всередині компонента.
- Для задіяного компонента запускається повторний рендеринг.
- Оновлений інтерфейс користувача генерується як новий віртуальний DOM.
- Diffing порівнює новий віртуальний DOM з попереднім.
- Справжній DOM оновлює лише змінені частини для кращої продуктивності.

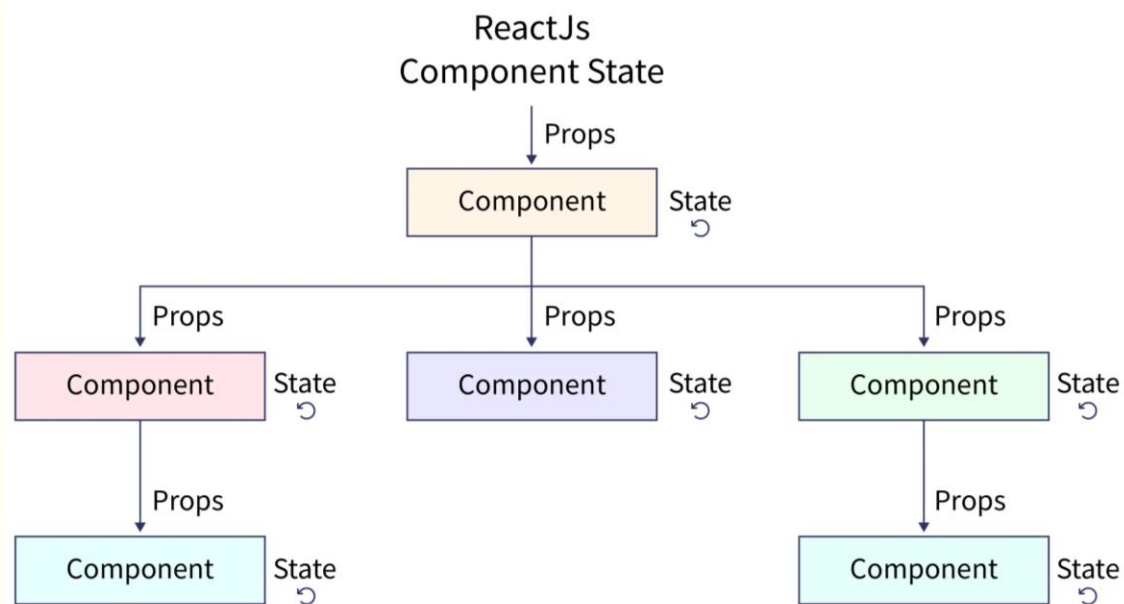
```
function Greet(props) {  
  return <h1>Hello, {props.name}</h1>;  
}
```

```
class Greet extends React.Component {  
  render() {  
    return <h1>Hello, {this.props.name}</h1>;  
  }  
}
```

- Типи компонентів React:

- *Функціональні компоненти* – це функції JavaScript, які повертають елементи React.
- *Компоненти класів* – це ES6-класи в React, які розширюють `React.Component` та підтримують обробку станів та життєвого циклу.

Керування станом компонента



```
import React, { useState } from 'react';

function Counter() {
  const [count, setCount] = useState(0);

  return (
    <div>
      <p>Count: {count}</p>
      <button onClick={() => setCount(count + 1)}>Increment</button>
    </div>
  );
}

export default Counter;
```

- Якщо props – це вхідні дані, що надходять до компонента, то state – це приватна пам'ять компонента, значення, якими він володіє, оновлює та використовує для рендерингу.
 - Стан є локальним для компонента і не може бути доступний дочірнім компонентам, якщо тільки він не переданий як props.
 - Він змінний, тобто може змінюватися з часом залежно від взаємодії з користувачем або відповідей API.
 - Коли стан оновлюється, компонент повторно рендериться, щоб відобразити зміни.
 - Керується за допомогою useState у функціональних компонентах або this.setState у компонентах класу.

Count: 0

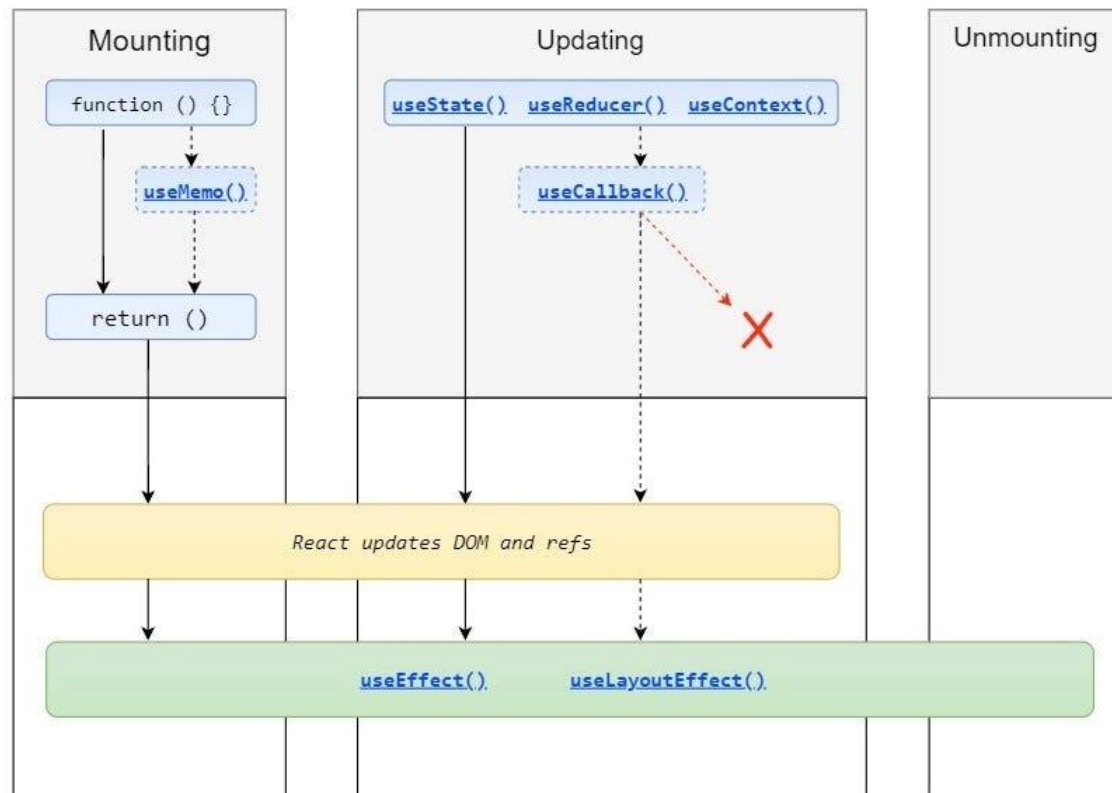
Increment

Керування станом компонента

React Hooks Lifecycle

"Render phase"

Pure and has no side effects. May be paused, aborted or restarted by React



"Commit phase"

Can work with DOM, run side effects, schedule updates.

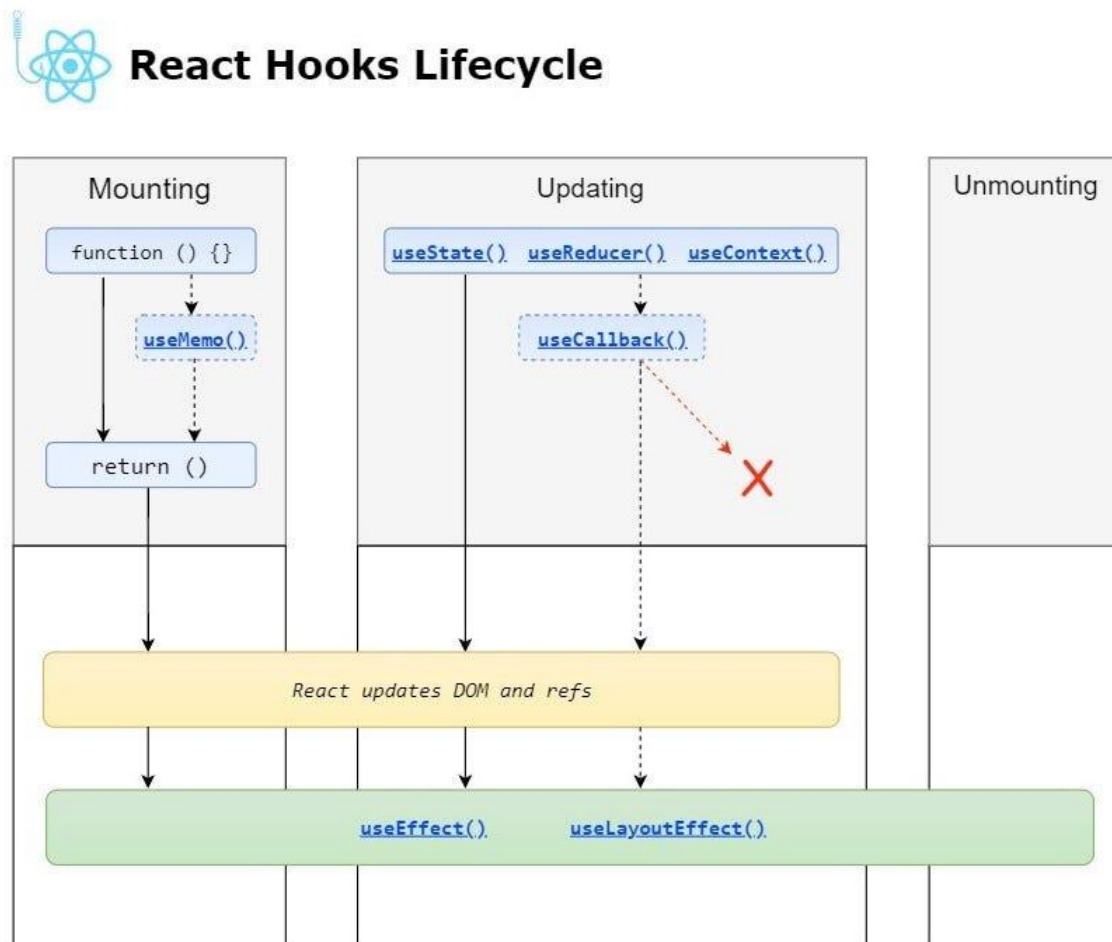


Props vs State

- ✓ props are read-only
- ✓ props can not be modified
- ✓ state changes can be asynchronous
- ✓ state can be modified using `this.setState`

- Хук React – це спеціальна функція, яка дозволяє вам «підключатися» до стану та життєвого циклу React з функціональних компонентів.
 - Хуки були введені в React 16.8, щоб функціональні компоненти могли мати стан та побічні ефекти, які раніше були доступні лише в компонентах класу.
 - Хуки React традиційно іменуються з префіксом "use", наприклад, `useState` або `useEffect`.
 - Хуки потрібно викликати на верхньому рівні функціонального компонента, а не всередині вкладених функцій, циклів або звичайних функцій JavaScript.
 - Для інкапсуляції та повторного використання логіки компонента можна створювати власні хуки, які дотримуються тих же правил.

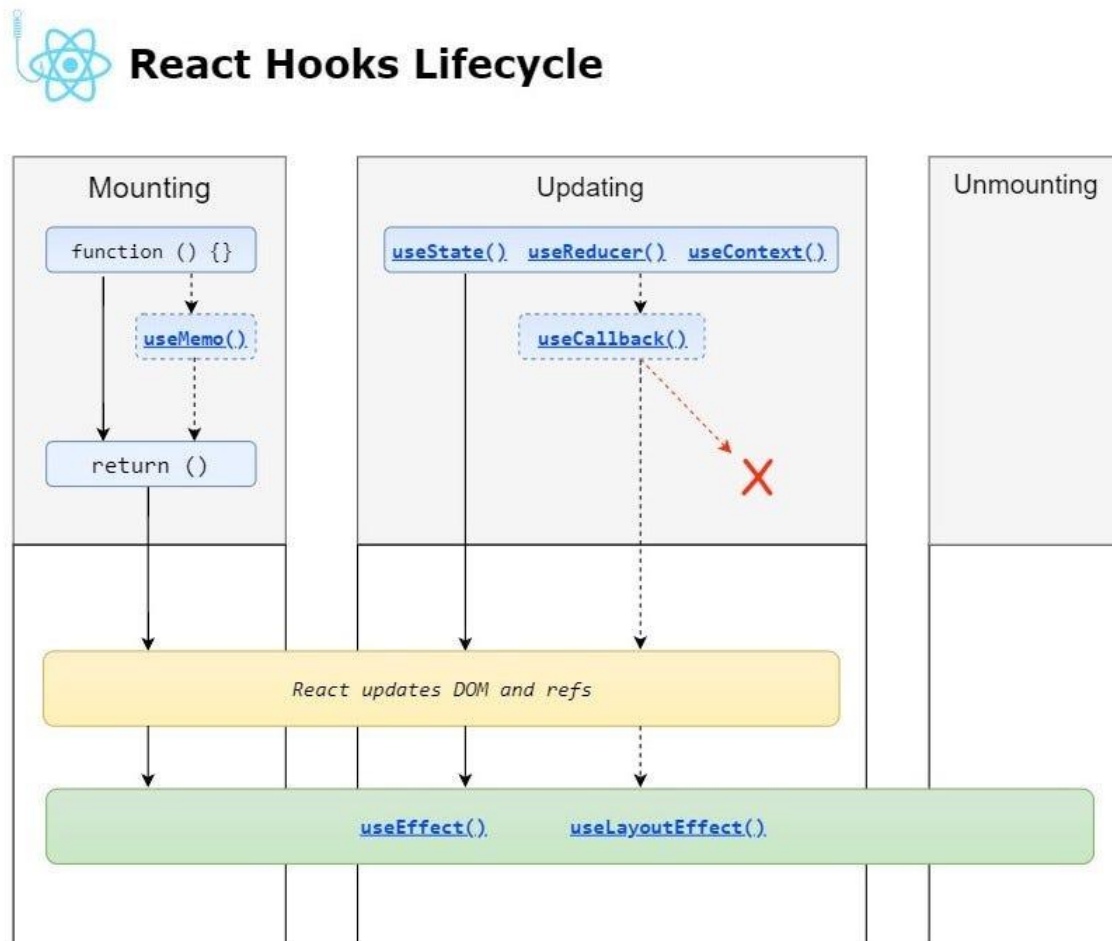
Керування станом компонента



Компоненти потужні як інструмент повторного використання, але вони повинні відображати певний UI, що робить їх незручними для спільного використання *невізуальної* логіки.

- Складні шаблони, такі як render props та компоненти вищого порядку, вимагають реструктуризації компонентів під час їх використання, що може бути громіздким і ускладнювати відстеження коду.
- Функції здаються ідеальним механізмом для повторного використання коду, проте вони не можуть мати локальний стан компонента React всередині себе.
- За допомогою хуків ви можете витягувати логіку стану з компонента, щоб її можна було незалежно відтестувати та використовувати повторно.
- Хуки дозволяють повторно використовувати логіку стану, не змінюючи ієрархію компонентів.

Керування станом компонента



React-хуки дозволяють розділити один компонент на менші функції на основі пов'язаних між собою частин (наприклад, налаштування підписки або отримання даних).

- Так ми можемо розділяти завдання додатка на основі логіки та методу життєвого циклу, до якого вони належать.

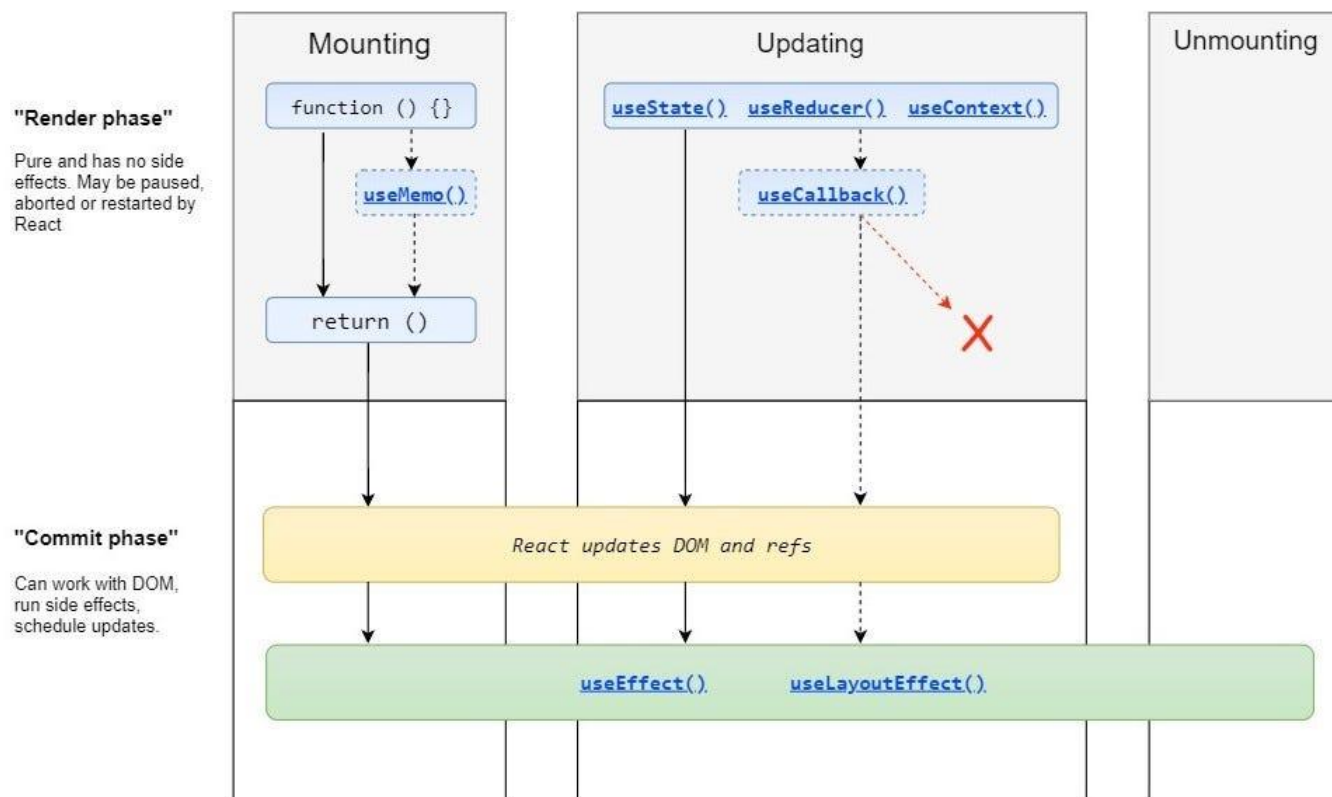
Починаючи з React 16.8, функціональні компоненти можуть мати стан за допомогою хуків, використовуючи метод `useState()`.

```
const [state, setState] = useState(initialState);
```

- Повертає значення стану та функцію для його оновлення.
- Під час початкового рендерингу повернутий стан (state) такий самий, як перший аргумент (initialState).

Керування станом компонента

React Hooks Lifecycle



```
const [state, setState] = useState(initialState);
```

Функція `setState()` використовується для оновлення стану.

- Вона приймає нове значення стану та ставить у чергу повторний рендеринг компонента.
- Ми можемо викликати цю функцію з будь-якого місця в компоненті, щоб оновити значення.
- Ця функція така ж, як і використання методу `this.setState()` у компоненті класу, за винятком того, що вона не об'єднує старий та новий стани.
- Під час наступних повторних рендерингів перше значення, яке повертається `useState()`, завжди буде найновішим станом після застосування оновлень.

Ми можемо багаторазово використовувати React-хуки в одному компоненті.

```
function declaringMultipleStateWithHooks () {  
  // Declare multiple state variables  
  const [name, setName] = useState('Ayush');  
  const [mobile, setMobile] = useState(8888999989);  
  const [address, setAddress] = useState({  
    street: '#1234, 14th Main Road, 7th Cross Road,  
    city: 'Bangalore', country: 'India'  
  });  
  // ..... (other code of component)  
}
```

```
handleName = () => {  
  setName('Ayush Verma')  
}
```


Керування станом компонента

```
import React, { useState } from 'react';

function Example() {
  // Оголошуємо нову змінну стану, яку назовемо "count"
  const [count, setCount] = useState(0);

  return (
    <div>
      <p>Ви натиснули {count} разів</p>

      <button onClick={() => setCount(count + 1)}>
        Натисни мене
      </button>
    </div>
  );
}

// Назви ліворуч (fruit, setFruit) не є частиною API React.
const [fruit, setFruit] = useState('banana');
/*
Цей синтаксис називається "деструктуризація масиву".
Ми створюємо дві нові змінні:
- fruit отримує перше значення, яке повертає useState()
- setFruit отримує друге значення
Еквівалентний запис виглядає так:
*/
var fruitStateVariable = useState('banana');
var fruit = fruitStateVariable[0]; // значення стану
var setFruit = fruitStateVariable[1]; // функція оновлення стану
```

- Якщо новий стан обчислюється з використанням попереднього стану, ви можете передати функцію до `setState`.
 - Функція отримає попереднє значення та поверне оновлене значення.

```
function Counter({initialCount}) {
  const [count, setCount] = useState(initialCount);
  return (
    <>
      Count: {count}
      <button onClick={() => setCount(initialCount)}>Reset</button>
      <button onClick={() => setCount(prevCount => prevCount - 1)}>
        - </button>
      <button onClick={() => setCount(prevCount => prevCount + 1)}>
        + </button>
    </>
  );
}
```

- Кнопки «+» та «-» використовують функціональну форму, оскільки оновлене значення базується на попередньому значенні.
- Але кнопка Reset використовує звичайну форму, оскільки вона завжди повертає лічильник до початкового значення.

Керування станом компонента. Lazy initial state

- Аргументом `initialState` є стан, який використовується під час початкового рендерингу.
 - У наступних рендерингах він ігнорується.
 - Якщо початковий стан є результатом ресурсоємного обчислення, ви можете замість нього надати функцію, яка буде виконана лише під час початкового рендерингу:

```
const [state, setState] = useState(() => {  
  const initialState = someExpensiveComputation(props);  
  return initialState;  
});
```

- Якщо ви оновите хук стану до того ж значення, що й поточний стан, React завершить роботу без рендерингу дочірніх об'єктів або спрацьовування ефектів.
 - React використовує алгоритм порівняння `Object.is`.

Керування станом компонента. Хук useEffect()

```
import React, { useState, useEffect } from "react";

function ExampleOfUseEffects() {
  // defining state variable
  const [name, setName] = useState({
    first: "Ayush", last: "Verma"
  });

  //using useEffect
  useEffect(() => {
    console.log("calling useEffect");
  });

  return (
    <div>
      <h3>First Name: {name.first}</h3>
      <h3>Last Name: {name.last}</h3>
    </div>
  );
}

export default ExampleOfUseEffects;
```

- Такі операції, як отримання даних з API, налаштування підписок та ручна зміна DOM у React Component, називаються «побічними ефектами» або скорочено ефектами, оскільки вони можуть впливати на інші компоненти та не можуть бути виконані до рендерингу.
 - Effects-хук useEffect додає можливість виконувати побічні ефекти з функціонального компонента.
 - Він виконує ту саму функцію, що й componentDidMount, componentDidUpdate та componentWillUnmount у класах React.
 - Ефекти оголошуються всередині компонента, тому вони мають доступ до його властивостей та стану.
 - Як і у випадку з useState, ви можете використовувати більше одного ефекту в компоненті.
- Використовуючи useEffect, ви повідомляєте React, що ваш компонент *має щось зробити після рендерингу*.
 - React запам'ятає передану вами функцію (ефект) та викличе її пізніше після виконання оновлень DOM.
 - За замовчуванням хук запускається як після першого рендерингу, так і *після кожного оновлення*.
 - React гарантує, що DOM буде оновлено на момент запуску ефектів.
 - Щоразу, коли ми повторно рендеримо, ми плануємо *інший* ефект, який замінює попередній.

```
import React, { useState, useEffect } from "react";

function ExampleOfUseEffects() {
  // Оголошення змінної стану
  const [name, setName] = useState({
    first: "Ayush",
    last: "Verma"
  });

  /*
   * useEffect перевіряє, чи змінилося значення змінної name.
   * Якщо name змінюється, виконується передана функція.
   */
  useEffect(() => {
    console.log(
      "Виклик useEffect при зміні значення змінної name"
    );

    setName({
      first: "Manav",
      last: "Yadav"
    });
  }, [name]); // Масив залежностей: ефект спрацює при зміні name

  return (
    <div>
      <h3>Ім'я: {name.first}</h3>
      <h3>Прізвище: {name.last}</h3>
    </div>
  );
}

export default ExampleOfUseEffects;
```

Керування станом компонента.

Хук useEffect()

- Метод `useEffect` поводитьься як метод `componentDidUpdate`.
 - Ми можемо зробити його умовним, коли викликати ефект.
 - Ми можемо передати масив як другий аргумент `useEffect`.
 - Цей масив повідомляє React, що потрібно просто викликати `useEffect` щоразу, коли поля в масиві змінюються
 - У коді він викличе ефект лише тоді, коли значення імені змінюється.

```
import React, { useState, useEffect } from "react";

function ExampleOfUseEffects() {
  // Оголошення змінної стану
  const [name, setName] = useState({
    first: "Ayush",
    last: "Verma"
  });

  // Використання useEffect
  useEffect(() => {
    console.log("Виклик useEffect лише один раз");
  }, []);

  /*
  Цей ефект також виконується лише один раз
  після першого монтування компонента.
  За поведінкою подібний до componentDidMount.
  */
  useEffect(() => {
    console.log(
      "Виклик useEffect аналогічно методу componentDidMount"
    );
  }, []);

  return (
    <div>
      <h3>Ім'я: {name.first}</h3>
      <h3>Прізвище: {name.last}</h3>
    </div>
  );
}

export default ExampleOfUseEffects;
```

Керування станом компонента.

Хук useEffect()

- Ми також можемо змусити useEffect поводитися подібно до componentDidMount, передавши порожній масив як другий аргумент useEffect.
 - Якщо ви передасте порожній масив ([]), властивості та стан всередині ефекту завжди матимуть свої початкові значення.
 - Хоча передача [] як другого аргументу ближча до звичної ментальної моделі componentDidMount та componentWillUnmount, зазвичай існують кращі рішення, щоб уникнути занадто частого повторного запуску ефектів.
 - Також не забувайте, що React відкладає запуск useEffect до того часу, як браузер завершить рендеринг, тому виконання додаткової роботи є меншою проблемою.


```
import React, { useState, useEffect } from "react";

function ExampleOfUseEffects() {
  // Оголошення змінної стану
  const [name, setName] = useState({
    first: "Ayush",
    last: "Verma"
  });

  /*
   * useEffect виконується та повертає функцію,
   * яка спрацює перед наступним запуском ефекту
   * або під час демонтування компонента.
   */
  useEffect(() => {
    console.log(
      "Виклик useEffect аналогічно методу componentWillUnmount"
    );

    return () => {
      console.log("Очищення...");
    };
  }, []);

  return (
    <div>
      <h3>Ім'я: {name.first}</h3>
      <h3>Прізвище: {name.last}</h3>
    </div>
  );
}

export default ExampleOfUseEffects;
```

Керування станом компонента. Хук `useEffect()`

Також можемо зробити так, щоб `useEffect` поведився подібно до методу `componentWillUnmount`.

- Функція очищення виконується перед видаленням компонента з інтерфейсу користувача, щоб запобігти витoku пам'яті.
- Крім того, якщо компонент рендериться кілька разів (як це зазвичай буває), попередній ефект очищається перед виконанням наступного ефекту.

Керування станом компонента. Час дії ефектів

- На відміну від `componentDidMount` та `componentDidUpdate`, функція, передана в `useEffect`, спрацьовує **після** `layout` та `paint`, під час відкладеної події.
 - Це робить її придатною для багатьох поширених побічних ефектів, таких як налаштування підписок та обробників подій, оскільки більшість типів роботи не повинні блокувати браузер від оновлення екрана.
 - Однак не всі ефекти можна відкласти.
 - Наприклад, видиме користувачеві змінювання DOM повинно спрацьовувати синхронно перед наступним рендерингом, щоб користувач не сприймав візуальної невідповідності.
 - Для цих типів ефектів React надає один додатковий хук під назвою `useLayoutEffect`.
 - Він має таку ж сигнатуру, як `useEffect`, і відрізняється лише тим, коли він спрацьовує.
 - Хоча `useEffect` відкладається до моменту завершення відображення в браузері, він гарантовано спрацює перед будь-якими новими рендерами.
 - React завжди скидатиме ефекти попереднього рендера перед початком нового оновлення.

Хук useRef()

```
import { useRef } from 'react';

function MyComponent() {
  const reference = useRef(initialValue);
  const someHandler = () => {

    // Access reference value:
    const value = reference.current;
    // Update reference value:
    reference.current = newValue;

  };

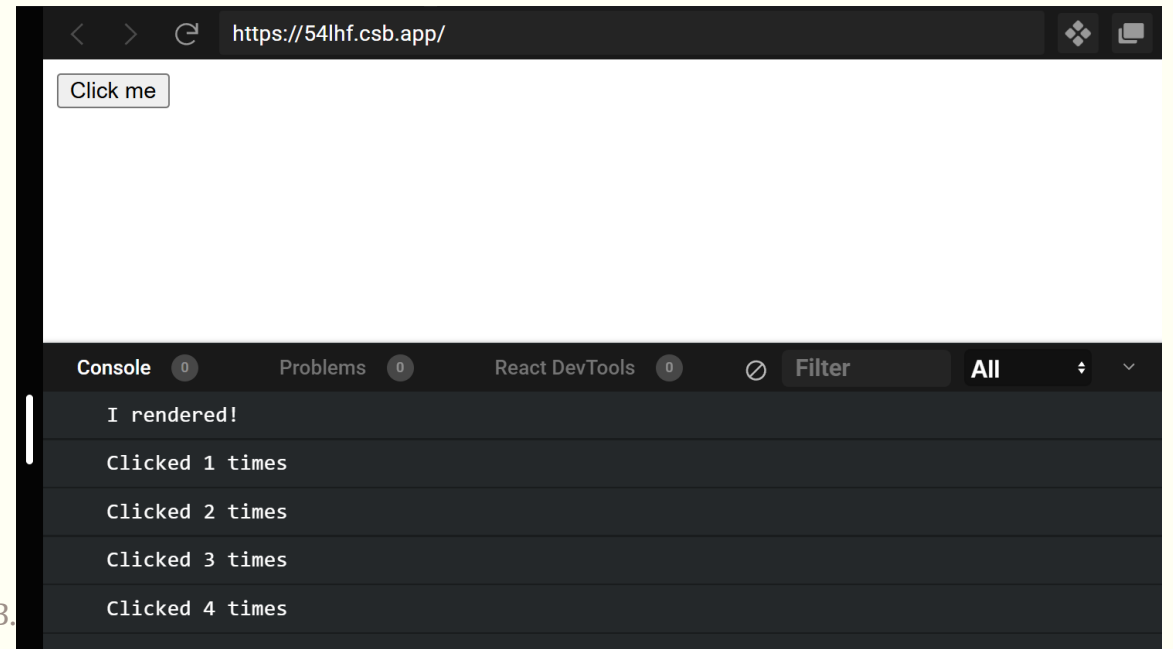
  // ...
}
```

- Хук `useRef`(хук) – це функція, яка повертає змінюваний (mutable) об'єкт `ref`, чия властивість `current` ініціалізується переданим аргументом (`initialValue`).
 - Повернений об'єкт зберігатиметься протягом усього терміну служби компонента.
 - `reference.current` отримує доступ до опорного значення, а `reference.current = newValue` оновлює його.
- Слід пам'ятати 2 правила:
 - Значення посилання *зберігається* (залишається незмінним) між повторними рендерингами компонентів;
 - Оновлення посилання *не призводить до повторного рендерингу* компонента.

Хук useRef()

```
1 import { useRef } from "react";
2
3 export default function LogButtonClicks() {
4   const countRef = useRef(0);
5
6   const handle = () => {
7     countRef.current++;
8     console.log(`Clicked ${countRef.current} times`);
9   };
10
11   console.log("I rendered!");
12
13   return <button onClick={handle}>Click me</button>;
14 }
```

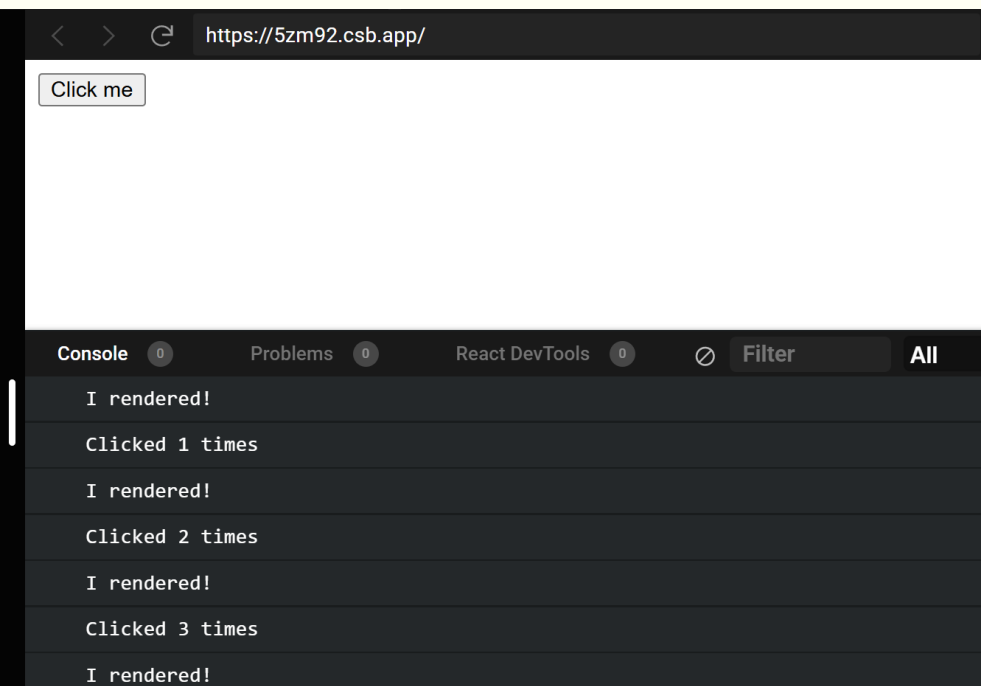
- Компонент LogButtonClicks використовує посилання для зберігання кількості кліків на кнопці.
 - `const countRef = useRef(0)` створює посилання `countRef`, ініціалізовані за допомогою 0.
 - Оновлення значення посилання `countRef.current++` не ініціює повторний рендеринг компонента: "I rendered!" записується в консоль лише один раз, під час початкового рендерингу, і жодного повторного рендерингу не відбувається, коли посилання оновлюється.



Різниця між посиланнями та станами

- Знову розглянемо компонент LogButtonClicks, але цього разу використаємо хук `useState()` для підрахунку кількості натискань кнопок.
 - Кожного разу, коли стан оновлюється, компонент повторно рендериться.
- 2 основні відмінності:
 - Оновлення посилання не ініціює повторний рендеринг, тоді як оновлення стану призводить до повторного рендерингу компонента;
 - Оновлення посилання є синхронним (оновлене значення посилання доступне одразу), тоді як оновлення стану є асинхронним (змінна стану оновлюється після повторного рендерингу).
- Посилання зберігають дані інфраструктури побічних ефектів, тоді як стан зберігає інформацію, яка безпосередньо відображається на екрані.

```
1 import { useState } from "react";
2
3 export default function LogButtonClicks() {
4   const [count, setCount] = useState(0);
5
6   const handle = () => {
7     const updatedCount = count + 1;
8     console.log(`Clicked ${updatedCount} times`);
9     setCount(updatedCount);
10  };
11
12  console.log("I rendered!");
13
14  return <button onClick={handle}>Click me</button>;
15 }
```



```
import { useRef, useState, useEffect } from 'react';
```

```
function Stopwatch() {  
  const timerIdRef = useRef(0);  
  const [count, setCount] = useState(0);  
  
  const startHandler = () => {  
    if (timerIdRef.current) { return; }  
    timerIdRef.current = setInterval(() => setCount(c => c+1), 1000);  
  };  
};
```

```
const stopHandler = () => {  
  clearInterval(timerIdRef.current);  
  timerIdRef.current = 0;  
};
```

```
const resetHandler = () => {  
  clearInterval(timerIdRef.current);  
  timerIdRef.current = 0;  
  setCount(0);  
};
```

```
useEffect(() => {  
  return () => clearInterval(timerIdRef.current);  
}, []);
```

```
return (  
  <div>  
    <div>Timer: {count}s</div>  
    <div>  
      <button onClick={startHandler}>Start</button>  
      <button onClick={stopHandler}>Stop</button>  
      <button onClick={resetHandler}>Reset</button>  
    </div>  
  </div>  
)  
);  
}
```

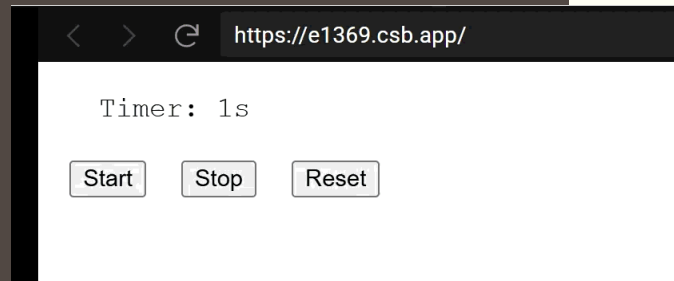
Приклад використання: реалізація секундоміра

- Ви можете зберігати всередині еталонної інфраструктури дані про побічні ефекти.

Наприклад, ви можете зберігати в еталонних вказівниках: ідентифікатори таймерів, ідентифікатори сокетів тощо.

Компонент Stopwatch використовує функцію таймера `setInterval(callback, time)` для збільшення лічильника секундоміра щосекунди.

- Ідентифікатор таймера зберігається у посиланні `timerIdRef`.
- Функція `startHandler()`, яка викликається при натисканні кнопки «Пуск», запускає таймер і зберігає ідентифікатор таймера у посиланні `timerIdRef.current = setInterval(...)`.
- Обробник кнопки «Stop», `stopHandler()`, отримує доступ до ідентифікатора таймера з посилання та зупиняє його `clearInterval(timerIdRef.current)`.
- Крім того, якщо компонент відмонтується з активним секундоміром, функція очищення `useEffect()` також зупинить таймер.



Хук useRef(). Доступ до елементів DOM

```
1 import './styles.css';
2 import { useRef, useEffect } from 'react';
3
4 export default function AccessingElement() {
5   const elementRef = useRef();
6
7   useEffect(() => {
8     const divElement = elementRef.current;
9     console.log(divElement);
10  }, []);
11
12  return <div ref={elementRef}>I'm an element</div>;
13 }
```

Виконується в 3 кроки:

- Визначити посилання для доступу до елемента `const elementRef = useRef();`
- Присвоїти посилання `ref` атрибуту елемента: `<div ref={elementRef}></div>`
- Після монтування `elementRef.current` вказує на елемент DOM.

Наприклад, потрібно буде отримати доступ до елементів DOM, щоб сфокусуватися на полі введення під час монтування компонента.

```
src > App.js > ...
1 import { useRef, useEffect } from 'react';
2
3 export default function InputFocus() {
4   const inputRef = useRef();
5
6   useEffect(() => {
7     inputRef.current.focus();
8   }, []);
9
10  return <input ref={inputRef} type="text" />;
11 }
```

< > ↻ <https://bcf4y.csb.app/>

- Щоб це запрацювало, потрібно створити посилання на вхід, призначити посилання `ref` атрибуту тегу, а після монтування викликати спеціальний метод `element.focus()` для елемента.

Хук useRef(). Доступ до елементів DOM

```
1 import { useRef, useEffect } from "react";
2
3 export default function InputFocus() {
4   const inputRef = useRef();
5
6   useEffect(() => {
7     // Logs `HTMLInputElement`
8     console.log(inputRef.current);
9
10    inputRef.current.focus();
11  }, []);
12
13  // Logs `undefined` during initial rendering
14  console.log(inputRef.current);
15
16  return <input ref={inputRef} type="text" />;
17 }
```

Під час початкового рендерингу посилання, яке має містити елемент DOM, є порожнім.

- Під час початкового рендерингу React все ще визначає, що буде виводом компонента, тому структура DOM ще не створена.
- Тому під час початкового рендерингу `inputRef.current` обчислюється значення `undefined`.

Хук `useEffect(callback, [])` здійснює зворотний виклик одразу після монтування, коли елемент `input` вже створено в DOM.

- Callback-функція в `useEffect(callback, [])` є правильним місцем для доступу до `inputRef.current`, оскільки гарантовано сконструйовано DOM.

Хук useRef(). Оновлення обмежень щодо посилань

```
import { useRef, useEffect } from 'react';

function MyComponent({ prop }) {
  const myRef = useRef(0);

  useEffect(() => {
    myRef.current++; // Good!
    setTimeout(() => {
      myRef.current++; // Good!
    }, 1000);
  }, []);

  const handler = () => {
    myRef.current++; // Good!
  };

  myRef.current++; // Bad!
  if (prop) {
    myRef.current++; // Bad!
  }

  return <button onClick={handler}>My button</button>
}
```

- Область видимості функції функціонального компонента повинна або обчислювати output, або викликати хуки.
 - Тому оновлення посилання (як і оновлення стану) не слід виконувати в межах безпосередньої області дії функції компонента.
 - Посилання має бути оновлене або всередині `useEffect()` зворотного виклику, або всередині обробників (обробників подій, обробників таймерів тощо).
- Ключові моменти, які слід пам'ятати під час використання хука `useRef`:
 - Посилання, створене за допомогою `useRef`, буде створено лише після того, як компонент буде змонтовано та збережено протягом повного життєвого циклу.
 - Посилання можна використовувати для доступу до вузлів DOM або елементів React, а також для зберігання мутабельних змінних (наприклад, `instance variables` у компонентах класу).
 - Оновлення посилання є побічним ефектом, тому його слід робити лише всередині `useEffect` (або `useLayoutEffect`) або всередині обробника подій.

Хук useContext() та Context API

```
import React from "react";

// Створення Context
const NumberContext = React.createContext();

// повертає об'єкт із значеннями { Provider, Consumer }
export default function App() {
  // Provider використовується для передачі значення
  // всім дочірнім і вкладеним компонентам
  return (
    <NumberContext.Provider value={42}>
      <div>
        <Display />
      </div>
    </NumberContext.Provider>
  );
}

function Display() {
  // Consumer використовується для отримання значення
  // з контексту. Цей компонент не отримує props.
  return (
    <NumberContext.Consumer>
      {(value) => (
        <div>Відповідь: {value}</div>
      )}
    </NumberContext.Consumer>
  );
}
```

- Хук `useContext` спрощує передачу даних по всьому застосунку без ручної передачі props вниз по дереву.
 - Цей метод є альтернативою перебору props по дереву компонентів та створює внутрішній глобальний стан для передачі даних.
 - Він є частиною React Context API (як і компоненти `Provider` та `Consumer`).
 - Context може стати простою альтернативою `Redux`, коли ваші дані прості або ваш додаток невеликий.
 - `useContext` просто робить речі зручнішими, не надаючи додаткових можливостей.
- Типовий спосіб використання Context API з `Consumer` представлено в коді.
 - Спочатку ми створюємо новий контекст, який зберігаємо в `NumberContext`. Отримана пара значень вміє взаємодіяти один з одним (але не з іншими контекстами).
 - Потім рендериться `NumberContext.Provider` з певним вмістом та передається `value` йому prop.
 - Це зробить це значення доступним для всіх його нащадків та їхніх нащадків. Усе піддерево зможе використовувати `Consumer` (або `useContext`) для зчитування значення.
 - Нарешті, зчитуємо значення за допомогою `Consumer` всередині `Display`-компонента.

Хук useContext() та Context API

```
import React from "react";

// Створення Context
const NumberContext = React.createContext();

// повертає об'єкт із значеннями { Provider, Consumer }
export default function App() {
  // Provider використовується для передачі значення
  // всім дочірнім і вкладеним компонентам
  return (
    <NumberContext.Provider value={42}>
      <div>
        <Display />
      </div>
    </NumberContext.Provider>
  );
}

function Display() {
  // Consumer використовується для отримання значення
  // з контексту. Цей компонент не отримує props.
  return (
    <NumberContext.Consumer>
      {(value) => (
        <div>Відповідь: {value}</div>
      )}
    </NumberContext.Consumer>
  );
}
```

- Деструктуризація контексту необов'язкова – досить часто її роблять під час створення:

```
const { Provider, Consumer } = React.createContext();
```

- Тут ми зверталися до властивостей явно як `NumberContext.Provider` та `NumberContext.Consumer`.
- Consumer вносить додаткову вкладеність: нам потрібно обгорнути наш контент у `NumberContext.Consumer` та використовувати шаблон `render props` (передаючи функцію як дочірню), щоб отримати значення та відобразити його.
 - Це працює добре, і `render props` може бути доречним шаблоном для передачі динамічних даних.
 - Проте це створює додаткове вкладення, когнітивні витрати і виглядає трохи дивно.

Хук useContext() та Context API

```
// import useContext (or we could write React.useContext)
import React, { useContext } from 'react';

// ...

function Display() {
  const value = useContext(NumberContext);
  return <div>The answer is {value}</div>;
}
```

```
function HeaderBar() {
  return (
    <CurrentUser.Consumer>
      {user =>
        <Notifications.Consumer>
          {notifications =>
            <header>
              Welcome back, {user.name}!
              You have {notifications.length} notifications.
            </header>
          }
        </Notifications.Consumer>
      }
    </CurrentUser.Consumer>
  );
}
```

■ **useContext дозволяє «використовувати» контекст без Consumer.**

- Перепишемо компонент Display з хуком useContext.
- Викличте useContext, передайте об'єкт контексту, який ви отримали з React.createContext, і виведеться значення.
- Єдине, на що слід звернути увагу: потрібно передавати весь об'єкт контексту useContext, а не лише Consumer!

■ У вас може бути випадок, коли вашому компоненту потрібно отримувати дані з кількох батьківських контекстів.

- У зразку коду дуже багато вкладень, щоб отримати лише 2 значення, тому замінимо на useContext:

```
function HeaderBar() {
  const user = useContext(CurrentUser);
  const notifications = useContext(Notifications);

  return (
    <header>
      Welcome back, {user.name}!
      You have {notifications.length} notifications.
    </header>
  );
}
```

Хук useCallback()

- Хук useCallback має конкретну функцію: уникнути непотрібних повторних рендерів у вашому коді.

```
useCallback(() => {  
  myCallbackFunction()  
}, [dependencies]);
```

- Передається inline callback та масив залежностей.
 - useCallback поверне мемоізовану версію зворотного виклику, яка змінюється лише тоді, коли змінилася одна із залежностей.
 - Це корисно під час передачі зворотних викликів оптимізованим дочірнім компонентам, які покладаються на рівність посилань, щоб запобігти непотрібним рендерингам (наприклад, shouldComponentUpdate).
- Ви також можете передати порожній масив залежностей.
 - Функція виконається лише один раз.
 - Якщо ви не передасте масив, вона повертатиме нове значення при кожному виклику.

```
useCallback(() => {  
  myCallbackFunction()  
}, []);
```



```
import React, { useState, useCallback } from "react";

const Counter = () => {
  const [count, setCount] = useState(0);
  const [otherCounter, setOtherCounter] = useState(0);

  const increment = () => {
    setCount(count + 1);
  };

  const decrement = () => {
    setCount(count - 1);
  };

  const incrementOtherCounter = () => {
    setOtherCounter(otherCounter + 1);
  };

  return (
    <>
      Count: {count}
      <button onClick={increment}>+</button>
      <button onClick={decrement}>-</button>
      <button onClick={incrementOtherCounter}>
        incrementOtherCounter
      </button>
    </>
  );
};

ReactDOM.render(
  <Counter />,
  document.getElementById("app")
);
```

Хук useCallback()

- Цей хук корисний, коли у вас є компонент з дочірнім елементом, який часто перерендериться, і ви передаєте йому зворотний виклик.
 - Проблема полягає в тому, що щоразу, коли лічильник оновлюється, всі 3 функції створюються знову.
 - Ви можете візуалізувати це, створивши екземпляр Set та додавши до нього кожну функцію.
 - Він зберігає лише унікальні елементи, що в нашому випадку означає різні (унікально створені екземпляри) функції.
- Якщо ви збільшите один лічильник, усі функції, пов'язані з цим лічильником, мають бути переініціалізовані.
- Якщо інше значення стану не змінюється, його не слід чіпати.
- У більшості випадків це не є великою проблемою.
- Хіба що ви передаєте багато різних функцій, які змінюють непов'язані фрагменти даних.

```
const Counter = () => {
  const [count, setCount] = useState(0)
  const [otherCounter, setOtherCounter] = useState(0)

  const increment = () => {
    setCount(count + 1)
  }
  const decrement = () => {
    setCount(count - 1)
  }
  const incrementOtherCounter = () => {
    setOtherCounter(otherCounter + 1)
  }

  functionsCounter.add(increment)
  functionsCounter.add(decrement)
  functionsCounter.add(incrementOtherCounter)

  console.log(functionsCounter.size);

  return (
    <>
      Count: {count}
    </>
  );
};
```

```
import React, { useState, useCallback } from "react";
```

```
export default function App() {
  return (
    <div className="App">
      <Counter />
    </div>
  );
}

const functionsCounter = new Set();

const Counter = () => {
  const [count, setCount] = useState(0);
  const [otherCounter, setOtherCounter] = useState(0);

  const increment = useCallback(() => { setCount(count + 1); }, [count]);
  const decrement = useCallback(() => { setCount(count - 1); }, [count]);
  const incrementOtherCounter = useCallback(() => {
    setOtherCounter(otherCounter + 1);
  }, [otherCounter]);

  functionsCounter.add(increment);
  functionsCounter.add(decrement);
  functionsCounter.add(incrementOtherCounter);

  console.log(functionsCounter);
  console.log(functionsCounter.size);

  return (
    <>
      Count: {count}
      <button onClick={increment}>+</button>
      <button onClick={decrement}>-</button>
      <button onClick={incrementOtherCounter}>incrementOtherCounter</button>
    </>
  );
};
```

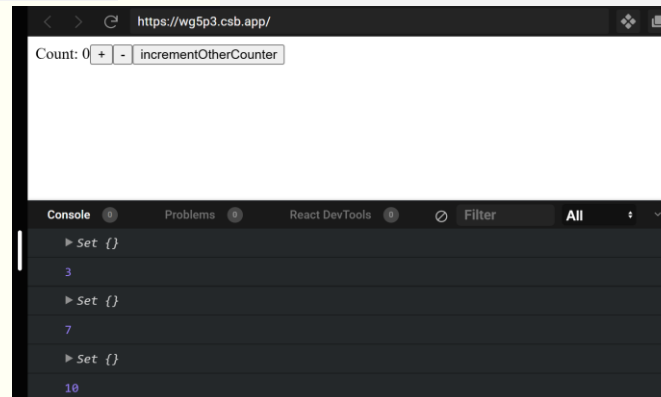
Хук useCallback()

- Якщо це проблема, ви можете скористатися useCallback.
- Переконайтеся, що ви додали цей масив як другий параметр до useCallback() потрібного стану.
- Тепер, якщо ви спробуєте клацнути один із лічильників, будуть повторно створені лише функції, пов'язані зі станом, який змінюється.

```
const increment = (() => {
  setCount(count + 1)
})
const decrement = (() => {
  setCount(count - 1)
})
const incrementOtherCounter = (() => {
  setOtherCounter(otherCounter + 1)
})
```



```
const increment = useCallback(() => {
  setCount(count + 1)
}, [count])
const decrement = useCallback(() => {
  setCount(count - 1)
}, [count])
const incrementOtherCounter = useCallback(() => {
  setOtherCounter(otherCounter + 1)
}, [otherCounter])
```



Хук useCallback()

- useCallback запам'ятовує не результат функції, а, швидше, сам об'єкт функції.

```
function ParentComponent() {  
  const onHandleClick = useCallback(() => {  
    const special = calculatePi();  
  });  
  
  return (  
    <SubComponent  
      handleClick={onHandleClick}  
    />  
  );  
}
```

- У цьому прикладі, щоразу, коли <SubComponent> запускає зворотний виклик onHandleClick через свою властивість handleClick, буде спрацьовувати (ймовірно, дорого!) функція calculatePi().
- Результат стрілкової функції не заmemoїзується, *лише посилання*.
- Якщо ми хочемо уникнути перерахунку числа π для кожного handleClick, краще запам'ятати його calculatePi() безпосередньо через useMemo():

```
const memoizedPi = useMemo( calculatePi () );
```

Хук useMemo()

- UseMemo – це хук, який використовується у функціональному компоненті react і повертає мемоізоване значення.
 - Мемоізація – це концепція, яка полягає в тому, що нам не потрібно переобчислювати функцію з заданим аргументом наступного разу, а вона повертатиме кешований результат.
 - Мемоізована функція запам'ятовує результати виводу для заданого набору вхідних даних.
 - Щоразу, коли в компоненті React стан та властивості не змінюють компонент, і компонент не рендериться повторно, він показує той самий вивід.
 - Хук useMemo використовується для покращення продуктивності в React-застосунку.
- useMemo() приймає 2 аргументи: функцію compute, яка обчислює результат, та масив dependencies.

```
const memoizedResult = useMemo(compute, dependencies);
```

 - Під час початкового рендерингу useMemo(compute, dependencies) викликає compute, запам'ятовує результат обчислення та повертає його компоненту.
 - Якщо під час наступних рендерингів залежності не змінюються, тоді useMemo() не викликає compute, а повертає мемоізоване значення.
 - Інакше, під час повторного рендерингу useMemo() викличе compute, заемоізує нове значення та поверне його.

Хук useMemo()

```
import './styles.css';

import { useState } from 'react';

export default function CalculateFactorial() {
  const [number, setNumber] = useState(1);
  const [inc, setInc] = useState(0);

  const factorial = factorialOf(number);

  const onChange = (event) => {
    setNumber(Number(event.target.value));
  };
  const onClick = () => setInc((i) => i + 1);

  return (
    <div>
      Factorial of
      <input type="number" value={number} onChange={onChange} />
      is {factorial}
      <button onClick={onClick}>Re-render</button>
    </div>
  );
}

function factorialOf(n) {
  console.log("factorialOf(n) called!");
  return n <= 0 ? 1 : n * factorialOf(n - 1);
}
```

- Якщо ваш зворотний виклик обчислення використовує значення властивостей або станів, обов'язково вкажіть ці значення як залежності:

```
const memoizedResult = useMemo(() => {
  return expensiveFunction(propA, propB);
}, [propA, propB]);
```

- Щоразу, коли ви змінюєте вхідне значення, факторіал обчислюється та 'factorialOf(n) called!' записується в КОНСОЛЬ.



Хук useMemo()

```
import './styles.css';

import { useState } from 'react';

export default function CalculateFactorial() {
  const [number, setNumber] = useState(1);
  const [inc, setInc] = useState(0);

  const factorial = factorialOf(number);

  const onChange = (event) => {
    setNumber(Number(event.target.value));
  };
  const onClick = () => setInc((i) => i + 1);

  return (
    <div>
      Factorial of
      <input type="number" value={number} onChange={onChange} />
      is {factorial}
      <button onClick={onClick}>Re-render</button>
    </div>
  );
}

function factorialOf(n) {
  console.log("factorialOf(n) called!");
  return n <= 0 ? 1 : n * factorialOf(n - 1);
}
```

- З іншого боку, кожного разу, коли ви натискаєте кнопку «Re-render», значення стану inc оновлюється.
 - Оновлення inc запускає повторний рендеринг <CalculateFactorial />.
 - Але, як вторинний ефект, під час повторного рендерингу факторіал знову перераховується – 'factorialOf(n) called!' реєструється в консолі.
- Використовуючи useMemo(() => factorialOf(number), [number]) замість простого factorialOf(number), React запам'ятовує обчислення факторіала.
 - Тепер, якщо натиснути кнопку «Re-render», 'factorialOf(n) called!' не реєструється в консолі.

useMemo() проти useCallback()

- Хук `useMemo` дуже схожий на `useCallback`, проте `useCallback` повертає мемоізований зворотний виклик, а `useMemo` – мемоізоване значення, результат цього виклику функції.
 - Випадок використання також відрізняється.
 - `useCallback` використовується для зворотних викликів, що передаються дочірнім компонентам.
 - `useCallback()`, порівняно з `useMemo()`, є більш спеціалізованим хуком, який запам'ятовує зворотні виклики:

```
import { useCallback } from 'react';

function MyComponent({ prop }) {
  const callback = () => {
    return 'Result';
  };
  const memoizedCallback = useCallback(callback, [prop]);
  return <ChildComponent callback={memoizedCallback} />;
}
```

```
import { useMemo } from 'react';

function MyComponent({ prop }) {
  const callback = () => {
    return 'Result';
  };
  const memoizedCallback = useMemo(() => callback, [prop]);
  return <ChildComponent callback={memoizedCallback} />;
}
```

- У наведеному прикладі `useCallback(() => {...}, [prop])` повертає той же екземпляр функції, якщо залежність `prop` однакова.
- Ви можете використовувати той же спосіб з `useMemo()` для запам'ятовування зворотних викликів.
- Використовуйте мемоізацію обережно. Хоч `useMemo()` може покращити продуктивність компонента, вам слід переконатися, що ви профілюєте компонент з хуком та без нього.
- Тільки після цього зробіть висновок, чи варта мемоізація. Неправильне використання мемоізації може негативно вплинути на продуктивність.

Хук useReducer()

```
const initialState = {count: 0};
function reducer(state, action) {
  switch (action.type) {
    case 'increment':
      return {count: state.count + 1};
    case 'decrement':
      return {count: state.count - 1};
    default:
      throw new Error();
  }
}
function Counter() {
  const [state, dispatch] = useReducer(reducer, initialState);
  return (
    <>
    Count: {state.count}
    <button onClick={() => dispatch({type: 'decrement'})}>-</button>
    <button onClick={() => dispatch({type: 'increment'})}>+</button>
    </>
  );
}
```

useReducer використовується для зберігання та оновлення станів, як і хук useState.

- Він приймає функцію reducer як перший параметр, а початковий стан як другий.
- useReducer повертає масив, що містить поточне значення стану, та функцію dispatch, якій можна передати дію та пізніше викликати.
- Це схоже на шаблон, який використовує Redux, але з кількома відмінностями.
- Наприклад, функція useReducer тісно пов'язана з певним редуктором. Ми відправляємо об'єкти дій лише до цього редуктора, тоді як у Redux функція відправлення надсилає об'єкт дії до сховища.
- Під час відправлення компонентам не потрібно знати редуктор, який оброблятиме дію.

```
const [state, dispatch] = useReducer(reducer, initialArg, init);
```

Хук useReducer(). Лінива ініціалізація

```
function init(initialCount) { return {count: initialCount};}
function reducer(state, action) {
  switch (action.type) {
    case 'increment':
      return {count: state.count + 1};
    case 'decrement':
      return {count: state.count - 1};
    case 'reset': return init(action.payload);    default:
      throw new Error();
  }
}

function Counter({initialCount}) {
  const [state, dispatch] = useReducer(reducer, initialCount,
init); return (
    <>
    Count: {state.count}
    <button
      onClick={() => dispatch({type: 'reset', payload:
initialCount})}> Reset
    </button>
    <button onClick={() => dispatch({type: 'decrement'})}>-</button>
    <button onClick={() => dispatch({type: 'increment'})}>+</button>
    </>
  );
}
```

- Існує два різні способи ініціалізації стану useReducer.
 - Найпростіший спосіб – передати початковий стан як другий аргумент:

```
const [state, dispatch] = useReducer(
  reducer,
  {count: initialCount}
);
```
 - Також можете створити початковий стан ліниво: для цього можна передати функцію init як третій аргумент.
 - Початковий стан буде встановлено на init(initialArg).
 - Це дозволяє витягти логіку для обчислення початкового стану поза редуктором.
 - Це також зручно для скидання стану пізніше у відповідь на дію.
 - Якщо ви повернете те саме значення з хука Reducer, що й поточний стан, React завершить роботу без рендерингу дочірніх об'єктів або запуску ефектів.

useState проти useReducer

- Хоча `useState` є базовим хуком для керування простим перетворенням станів та додатковим хуком `useReducer` для керування складнішою логікою станів, `useState` всередині використовує `useReducer`.
 - Тобто ви можете використовувати `useReducer` для всього, що ви можете зробити з `useState`.
 - Однак, між цими двома хуками є деякі суттєві відмінності.
 - `useReducer` дозволяє уникнути передачі зворотних викликів через різні рівні вашого компонента.
 - Натомість даватиме можливість передати надану функцію `dispatch`, що, у свою чергу, покращить продуктивність компонентів, які запускають глибокі оновлення.
 - Коли є складна логіка для оновлення стану, ви просто не використовуватимете сеттер безпосередньо для оновлення стану, а напишете складну функцію, яка, у свою чергу, викличе сеттер з оновленим станом.
 - Тому рекомендується використовувати `useReducer`, який повертає метод `dispatch`, що не змінюється між повторними рендерами, і ви можете тримати логіку маніпуляцій у редукторах.

```
import { useState, useRef, forwardRef, useImperativeHandle } from "react";

const ChildOne = forwardRef((props, ref) => {
  const [count, setCount] = useState(0);

  useImperativeHandle(ref, () => ({
    count,
  }));

  const updateCount = () => {
    setCount((c) => c + 1);
    console.log(count + 1);
  };

  return <button onClick={updateCount}>Increment</button>;
});

const ChildTwo = forwardRef((props, ref) => {
  const checkCount = () => console.log("->", ref.current.count);

  return <button onClick={checkCount}>Count</button>;
});

const Parent = () => {
  const ref = useRef();

  return (
    <div>
      <ChildOne ref={ref} />
      <ChildTwo ref={ref} />
    </div>
  );
};

export default Parent;
```

Хук useImperativeHandle()

- useImperativeHandle дозволяє передавати значення та функції з дочірнього компонента назад до батьківського за допомогою ref.

- Звідти батьківський компонент може або використовувати їх сам, або передати їх *іншому дочірньому компоненту*.

useImperativeHandle(ref, createHandle, [deps])

- Ви можете передавати посилання як prop дочірньому об'єкту лише так, щоб його компонент обгортався в forwardRef.
- У коді ми ініціалізуємо ref в батьківському об'єкті, а потім передаємо його як властивість обом дочірнім об'єктам.
- Обидва дочірні об'єкти обгортають своє повернене значення forwardRef, щоб мати змогу приймати посилання як властивість.
- Перший аргумент, який приймає forwardRef, є звичайним props, а другий аргумент – ref.

```
import { useState, useRef, forwardRef, useImperativeHandle } from "react";
```

```
const ChildOne = forwardRef((props, ref) => {
  const [count, setCount] = useState(0);

  useImperativeHandle(ref, () => ({
    count,
  }));

  const updateCount = () => {
    setCount((c) => c + 1);
    console.log(count + 1);
  };

  return <button onClick={updateCount}>Increment</button>;
});

const ChildTwo = forwardRef((props, ref) => {
  const checkCount = () => console.log("->", ref.current.count);

  return <button onClick={checkCount}>Count</button>;
});

const Parent = () => {
  const ref = useRef();

  return (
    <div>
      <ChildOne ref={ref} />
      <ChildTwo ref={ref} />
    </div>
  );
};
```

```
export default Parent;
```

Хук useImperativeHandle()

- Тепер ChildOne має свій власний внутрішній стан, який керується за допомогою хука useState, і має метод updateCount, який збільшує значення count.
 - Потім він робить значення count доступним для Parent за допомогою хука useImperativeHandle, який приймає змінну ref як аргумент та повертає її count.
 - Parent тепер може отримати доступ до count та передає його як властивість ChildTwo, який при натисканні записує дані ref.current.count в консоль.
 - Оскільки ми використовуємо useRef, повторний рендеринг у Parent не відбувається.

Хук `useLayoutEffect()`

```
import React, { useLayoutEffect, useState } from 'react';

const App = () => {
  const [name, setName] = useState('Ayush');

  useLayoutEffect(() => {
    if (name === 'Ayush') {
      setName('Akash');
    }
  }, [name]);

  return <div> {name} has email id of ayush@medium.com </div>;
};

export default App;
```

- `useLayoutEffect` спрацьовує після завершення життєвого циклу рендерингу та безпосередньо перед виконанням `useEffect`.
 - Це корисно для синхронного повторного рендерингу DOM, а також для зчитування макета з DOM.
 - Але щоб запобігти блокуванню завантаження сторінки, нам завжди слід використовувати хук `useEffect`.
 - Його синтаксис ідентичний синтаксису `useEffect`.
- Якщо бути точнішим, `useLayoutEffect` спрацьовує *безпосередньо перед тим, як DOM має можливість рисувати*.
 - Це гарне місце для керування фокусом, виділенням тексту, тригерної анімації та інтеграції зі сторонніми DOM-бібліотеками.
 - Рекомендується спочатку спробувати використати `useEffect`, а якщо проблеми з інтерфейсом користувача все ще виникають, тоді звернутися до `useLayoutEffect`.

Хук `useLayoutEffect()`

- Варто зазначити, якщо ви використовуєте фреймворк на кшталт Next.js, він може генерувати HTML на *сервері*, тому видасть попередження, коли побачить `useLayoutEffect`, оскільки насправді він має працювати лише на *клієнті*.
 - `useLayoutEffect` нічого робити на сервері, оскільки його ефект не може бути закодований у вихідний формат рендерера сервера.
 - Це призведе до невідповідності між початковим UI та запланованим (intended) UI.
 - Щоб уникнути цього, `useLayoutEffect` слід використовувати лише в компонентах, які рендеряться виключно на клієнті.
- Кілька лайфхаків, щоб досягти цього:
 - або
- Ще один особливий випадок: ви оновлюєте значення (наприклад, `ref`) і хочете переконатися, що воно актуальне, перш ніж виконувати будь-який інший код.

```
if (typeof window !== "undefined")
  useLayoutEffect(() => console.log("I only run on the Client!"))
```

```
if (browser.process)
  useLayoutEffect(() => console.log("I only run on the Client!"))
```

```
const ref = React.useRef()

React.useEffect(() => {
  ref.current = 'some value'
})

// then, later in another hook or something

React.useLayoutEffect(() => {
  console.log(ref.current)
  // <-- this logs an old value because this runs first!
})
```

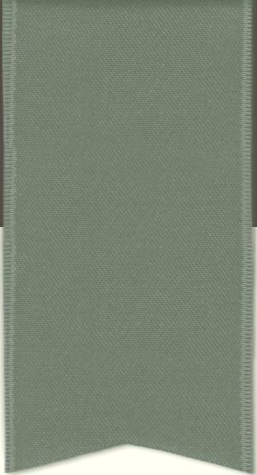

Хук `useDebugValue()`

- Цей хук надає користувацькі мітки користувацьким хукам, щоб зробити процес налагодження простішим та ефективнішим.
 - Він викликається лише тоді, коли React DevTools увімкнені. `useDebugValue(value)`
- ```
function useCustomHook(val) {
 const [value, setValue] = useState(null);
 useDebugValue(value ? Not Empty : Empty);
 return value;
}
```
- В React DevTools поруч із `useCustomHook` буде показано мітку.
- Наприклад, якщо значення `state` — «empty», тоді мітка матиме значення «Empty».
- У деяких випадках форматування значення для відображення може бути дорогим.
  - Воно також не потрібне, якщо хук фактично не перевіряється.
  - Тому `useDebugValue` приймає функцію форматування як необов'язковий другий параметр.
  - Ця функція викликається лише тоді, коли перевірено хуки. Вона отримує значення налагодження як параметр і повинна повертати відформатоване значення відображення.
  - Наприклад, користувацький хук, який повертає Date-значення, може уникнути непотрібного виклику функції `toDateString`, передавши такий формувальник:

# Важливі правила для хуків

---

- Викликайте хуки *лише на верхньому рівні вашого компонента*.
  - Не викликайте хуки зсередини циклу, умови або вкладених функцій.
  - Натомість завжди використовуйте хуки на верхньому рівні вашої функції React, перед будь-якими ранніми поверненнями. Так ви гарантуєте, що хуки викликатимуться в одному й тому ж порядку щоразу, коли компонент рендериться.
  - Це дозволяє React правильно зберігати стан хуків між кількома викликами методів `useState` та `useEffect`.
- Викликайте хуки *лише з функціональних компонентів React*.
  - Не викликайте хук зі звичайних функцій JavaScript.
  - Хуки можуть викликати інші хуки, тобто з користувацьких хуків.
  - Так ви гарантуєте, що вся логіка стану в компоненті чітко видима з його первинного коду.



# ДЯКУЮ ЗА УВАГУ!

Наступне питання: JavaScript-бібліотеки для розширеного керування станом